

24

Symbolic Fusion in Visual SLAM: A Hidden Markov Model Approach

Oscar Fuentes

Facultad de Ingeniería, Universidad Nacional Autónoma de México

Jesús Savage

Facultad Ingeniería, Universidad Nacional Autónoma de México

Marco Negrete

Facultad Ingeniería, Universidad Nacional Autónoma de México

CONTENTS

24.1	Introduction	430
24.2	Related Work	431
24.2.1	Particle Filtering	431
24.2.2	Graph-SLAM	431
24.2.3	Multi-Sensor Fusion SLAM	432
24.2.4	Summary	432
24.3	Proposed Methodology	432
24.3.1	Hidden Markov Models (HMMs) for SLAM	434
24.3.1.1	Model Definition	434
24.3.1.2	Symbolic Representation of Sensor Data	435
24.3.1.3	Localization with HMMs	436
24.3.1.4	Computational Efficiency	436
24.3.1.5	Adaptability to Sensor Fusion	436
24.3.1.6	Localization Process	436
24.3.2	Training the Hidden Markov Model (HMM)	437
24.3.2.1	Sensor Data Preprocessing	438
24.3.2.2	Model Initialization	439
24.3.3	Model Improvement-OnLine Baum Welch Algorithm ..	439
24.3.4	Benchmarking SLAM: Challenges and Considerations ..	440
24.4	Experiments	440
24.4.1	Training and Validation	441
24.4.2	Experimental Setup	443
24.4.3	Results	444
24.5	Conclusions	446

Acknowledgements	447
------------------------	-----

This work presents a lightweight and versatile SLAM system leveraging vision-based sensor inputs and Hidden Markov Models (HMMs) to construct a topological graph-based map of the environment. Our approach introduces a novel symbolic fusion technique that combines visual and LiDAR data through vector quantization, improving adaptability to dynamic and uncertain environments. By discretizing sensor readings into symbolic representations, the system facilitates efficient sensor fusion and robust localization, overcoming the limitations of single-sensor methods.

24.1 Introduction

In recent years, autonomous systems such as service robots and autonomous vehicles have become increasingly integrated into our daily lives, requiring reliable capabilities to explore, map, and navigate complex environments. Achieving these capabilities means handling diverse sensor data under varied environmental conditions. Simultaneous localization and mapping (SLAM) is the solution to this problem.

In this paper, we propose a graph-based lightweight SLAM system based on Hidden Markov Models (HMM) and vector quantization. Our system [801] is designed to take advantage of multiple sensor modalities. Clustering sensor readings into symbolic representations allows our system to achieve effective sensor fusion and adaptability in diverse environments, overcoming some of the primary limitations of traditional single-sensor approaches [802], [803].

Despite the progress made in SLAM, real-world applications continue to present significant challenges. Dynamic environments, such as those encountered by mobile robots in populated areas, require systems that can quickly adapt to changing conditions. Furthermore, uncertainty in sensor measurements—whether from noise, sensor drift, or environmental variables—adds complexity to the problem of achieving accurate localization and mapping. As a result, developing systems that can address these challenges in real-time remains a critical area of research.

Existing SLAM systems often rely on a single sensor or sensor fusion method requiring extensive calibration. For example, while traditional vision-based SLAM methods provide rich environmental details, they can be unreliable in low-light or feature-less environments. In contrast, our proposed method utilizes Hidden Markov Models (HMMs) and vector quantization to create a modular and flexible SLAM solution that effectively combines data from various sensors. This fusion of symbolic representations allows the system to handle dynamic and uncertain environments, and the model aims to learn

the most reliable inputs given specific areas or tasks. Our approach combines the strengths of multiple sensor modalities utilizing a symbolic fusion method [804]. The results of our research suggest that HMMs offer an adaptable, scalable, and reliable SLAM solution for dynamic and uncertain environments.

Our main contributions can be summarized as follows: 1) A graph SLAM system based on Hidden Markov Models, 2) a modular system capable of using a wide variety of sensors and features, 3) an autonomous training method, 4) a navigation method capable of avoiding obstacles, 5) a robust localization method and 6) a system with complexity in the order of $O(N^2T)$.

24.2 Related Work

The state of the art in SLAM remains largely based on three key paradigms: Kalman Filters, Particle Filters, and Graph-Based SLAM. Although the foundational principles of these methods continue to be integral to SLAM applications, recent advances have focused primarily on fine-tuning these approaches for specific robotic platforms and environments. In this work, we leverage the existing methodologies but build on them by incorporating novel sensor fusion techniques, drawing on the techniques of Particle Filtering [805] and Hidden Markov Models [806] for improved adaptability and robustness.

24.2.1 Particle Filtering

Particle Filtering, often implemented with **Rao-Blackwellized Particle Filters (RBPF)** [805], represents an important SLAM method that builds an occupancy grid map. The robot's localization process relies on filtering techniques akin to those in EKF, but with the added flexibility of particle filters. A key contribution to the development of this approach was made by Murphy et al. [807], who proposed the Rao-Blackwellized particle filter for SLAM. The joint posteriori is given as:

$$P(\mathbf{x}_t, m | \mathbf{z}_t, \mathbf{u}_{t-1}) = P(m | \mathbf{x}_t, \mathbf{z}_t) P(\mathbf{x}_t | \mathbf{z}_t, \mathbf{u}_{t-1}) \quad (24.1)$$

where $\mathbf{x}$ is the robot pose, m is the map, $\mathbf{z}_t$ are the observations, and $\mathbf{u}_{t-1}$ is the control signal.

The Rao-Blackwellized Particle Filter uses importance sampling filters, such as **Sequential Importance Resampling (SIR)**, to estimate the posterior distribution:

$$P(\mathbf{x}_1 : t | \mathbf{z}_1 : t, \mathbf{u}_{1:t-1}) \quad (24.2)$$

Each particle represents a possible robot trajectory, and an individual map is associated with each sample, constructed from observations [808].

24.2.2 Graph-SLAM

Graph-based SLAM, as proposed by Grisetti et al. [809], represents another effective solution to the SLAM problem. In this approach, robot poses at different time steps are represented as **nodes**, while the **edges** represent the constraints between poses. The SLAM problem is then solved by maximizing the **posterior probability**:

$$P(\mathbf{X}_t, M | \mathbf{Y}_t, \mathbf{U}_t, \mathbf{x}_0) \quad (24.3)$$

where $\mathbf{X}_t$ represents the poses of the robot, M is the map, $\mathbf{Y}_t$ are the sensor readings, and $\mathbf{U}_t$ is the motion of the robot.

Graph-SLAM has received attention for its ability to incorporate **loop closures**, where non-contiguous nodes are connected based on recognition of previously visited locations. This ensures that the graph remains consistent and accurate as new data is added. Notable work on **active SLAM** using topological graphs can be found in [810], where prior environmental information is integrated for efficient mapping and exploration.

24.2.3 Multi-Sensor Fusion SLAM

Multi-sensor fusion SLAM has emerged as a reliable solution to address the challenges of simultaneous localization and mapping in dynamic environments. By combining data from multiple sensors, such as IMUs, monocular cameras, and LiDAR, multisensor fusion SLAM can mitigate issues like sensor noise, sensor drift, and environmental variability, ultimately improving the accuracy and robustness of localization and mapping. The benefits of multi-sensor fusion SLAM are particularly evident in environments with degenerate geometries, where single-sensor approaches often struggle. LiDAR-visual-inertial fusion algorithms, for instance, have demonstrated improved performance in challenging environments by leveraging the complementary strengths of LiDAR, visual, and inertial sensors. When selecting a multi-sensor fusion approach, it is essential to consider the specific requirements of the task or environment.

24.2.4 Summary

Each SLAM approach presents advantages depending on the robot architecture and task at hand. Extended Kalman Filter (EKF SLAM) is most common practice with moderate uncertainty, while Particle Filtering excels in highly uncertain environments. Graph-SLAM is well-suited for large-scale environments, especially with loop closure. ROS-based implementations like Gmapping, Hector, and Cartographer provide accessible solutions for various robotic platforms. Our approach combines these methodologies' strengths to create a versatile system that can adapt to diverse robotic tasks.

24.3 Proposed Methodology

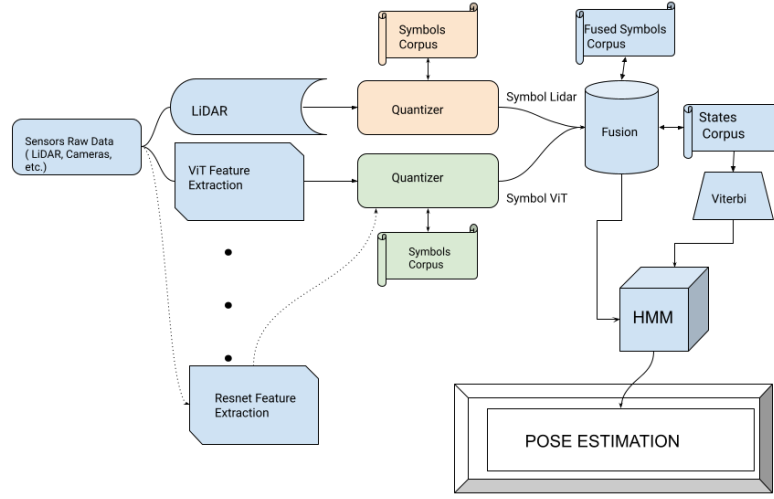


FIGURE 24.1
Proposed system pipeline.

The SLAM pipeline we propose is entirely characterized by the observation and transition matrices derived from the HMM training algorithms and the symbols used to represent observations. At this stage, the observations from various sensors, or their fusion, are represented by discrete symbols corresponding to similar sensor readings. To achieve this, we use vector quantization, wherein clustering algorithms such as MiniBatch K-Means [811] or Affinity Propagation [812] are employed to group similar sensor readings. Each resulting cluster is mapped to a unique symbol, enabling efficient representation and compression of the sensor data. This process facilitates symbolic fusion, effectively integrating different sensor modalities, which is essential for the discrete HMMs used in the SLAM pipeline proposed. A diagram showing these ideas can be seen in Figure 24.1.

Once the current reading is characterized as a symbol, we begin filling a buffer called the Viterbi Buffer. This buffer is then fed into the Viterbi algorithm [813], enabling an efficient stochastic estimation of the hidden states most likely to have generated the observations vector, which consists of the

last N readings stored in the buffer. The length of this buffer is an important hyperparameter that requires careful tuning, as the transition rate between hidden states can vary significantly across different environments, sensors, and feature extraction techniques, particularly in the vision-based implementation of the algorithm, where visual features from ResNet [814] or Vision Transformer DeiT [815] models play a critical role in symbol extraction. Visual data, processed by both ResNet and DeiT vision transformer, provides a rich feature set representing the environment, which is then quantized into discrete symbols using the aforementioned clustering techniques. Simultaneously, LiDAR data is also discretized into its own set of symbols. These visual and LiDAR-based symbols are then fused into a combined symbolic representation, allowing us to integrate the complementary information provided by the different sensors. It is important to mention that the Forward Algorithm provides a fast and efficient way to obtain the likelihood for a particular reading according to each model, one of the three HMM problems [802]. Finally, the hidden state sequence obtained is mapped back to a pose estimate. In our previous work [801], we proposed a method that refines pose estimation using ICP algorithms, applying affine transformations based on quantized LiDAR data. Although this paper focuses on the hidden state, representing a generalized quantized area, the previous approach could still be employed here if greater precision is needed, albeit at the cost of increased computational expenses. In contrast, we adopt a more computationally efficient method for this work. Specifically, we adjust the wheel odometry to the midpoint between the estimated states during transitions that exceed a reliability threshold. In the experiments section, we report the odometries obtained in this manner. Since the Euclidean distance between states in the state space is small, the quantization error introduced by this simple correction method can be considered negligible.

24.3.1 Hidden Markov Models (HMMs) for SLAM

In our approach, we use a Hidden Markov Model (HMM)-based framework for Simultaneous Localization and Mapping (SLAM), where the goal is to estimate a topographic graph based on noisy sensor measurements and pose estimates. The HMM consists of two random variables: a hidden state representing the robot's pose, and an observable variable derived from sensor readings, which are symbolically quantized to facilitate efficient data processing.

24.3.1.1 Model Definition

An HMM is defined by three components:

- **Transition Matrix ($\mathbf{A}$):** The transition probabilities between states, which describe the system's dynamics.

$$a_{ij} = P(s_t = s_j | s_{t-1} = s_i) \quad (24.4)$$

- **Emission Matrix (B):** The probability of observing a symbol O_t given a state s_t .

$$b_j(k) = P(O_t = V_k | s_t = s_j) \quad (24.5)$$

- **Initial State Distribution (π):** The initial probability distribution over the states.

Together, these components form the model $\lambda = [A, B, \pi]$.

24.3.1.2 Symbolic Representation of Sensor Data

To represent sensor readings, we discretize the sensor measurements through vector quantization. Visual data, processed by a CNN model such as ResNet, and DeIt ViT provides a rich feature set that is quantized into discrete symbols. Similarly, other sensor readings (e.g., LiDAR, IMU) are also discretized using clustering techniques such as MiniBatch K-Means or Affinity Propagation. Each resulting cluster is assigned a unique symbol, which serves as observations for the HMM. This approach enables the efficient fusion of sensor modalities like LiDAR and visual data (from cameras), which is essential for the versatility and adaptability of the SLAM system. We have tried different algorithms, each performing better at certain conditions. The K-Means Clustering algorithms often serve as a starting point, due to their efficiency in processing big volumes of data, however, they tend to skew the centroids due to repetition, which causes some “sensor degradation” as showed on Figure 24.2(a).

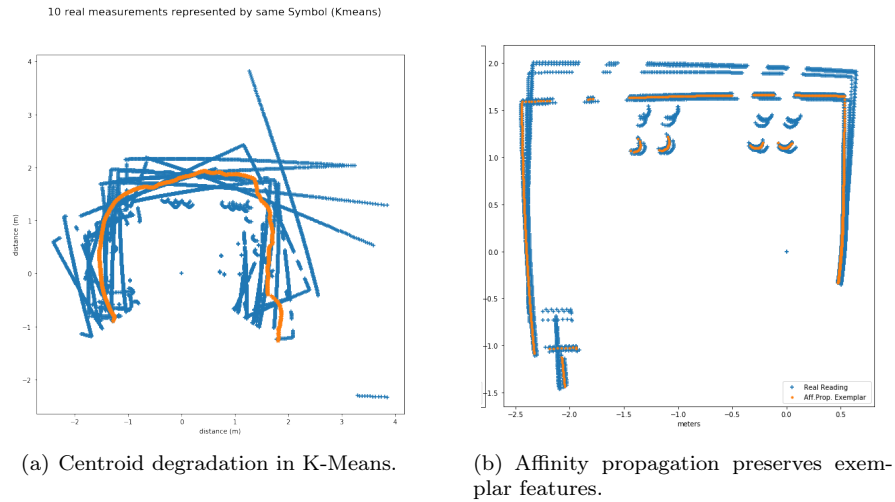


FIGURE 24.2

Clustering on the same dataset using K-Means (a) and Affinity Propagation (b).

The degradation of this data causes the centroids to gradually lose their ability to adequately represent the readings within the cluster. While the significantly more computationally expensive Affinity Propagation algorithm greatly mitigates this undesired effect (see Figure 24.2(b)), the impact is typically negligible for small training sets, such as those reported in the experiments section.

24.3.1.3 Localization with HMMs

Given a sequence of sensor observations, the HMM-based approach estimates the most probable sequence of robot poses. The Viterbi algorithm is employed to find the most likely hidden state sequence, which corresponds to the robot's path. The state space is defined as a discrete set of possible poses $\{s_1, s_2, \dots, s_N\}$, where each state represents a specific location and orientation of the robot. The observation set $\{V_1, V_2, \dots, V_K\}$ represents the quantized sensor data. The localization process can be expressed as:

$$P(O|\lambda) = \prod_{s=1}^N \pi_s \prod_{t=1}^T a_{st, s_{t+1}} b_{s_{t+1}}(o_t) \quad (24.6)$$

where O represents the sequence of observations, and a , b , and π are the transition, emission, and initial state probability matrices, respectively.

24.3.1.4 Computational Efficiency

To enhance computational efficiency, we use dynamic programming-based algorithms such as the Forward Algorithm and the Viterbi Algorithm. While the number of states N in the model affects computational complexity, dynamic programming reduces the time complexity from $O(N^T)$ (without optimization) to $O(N^2T)$, making the process feasible even with larger state spaces. The model's performance can be fine-tuned by adjusting the number of states in the system and the size of the observation codebook. A larger number of states results in a finer resolution of the map, which increases accuracy but at the cost of higher computational demands.

24.3.1.5 Adaptability to Sensor Fusion

In this work, we specifically focus on fusing the feature vectors extracted from a ResNet-50 CNN [814] and/or a ViT DeiT [?] model with LiDAR readings, however, many other feature extraction techniques might be better on a given environment, conditions, or even from the computational cost perspective. Visual data, processed by the ResNet or ViT model, provides a rich feature set representing the environment, which is then quantized into discrete symbols using the aforementioned clustering techniques. Simultaneously, LiDAR data is also discretized into its own set of symbols. These visual and LiDAR-based symbols are then fused into a combined symbolic representation, allowing us to integrate the complementary information provided by the different sensors.

24.3.1.6 Localization Process

For localization, we apply the Viterbi algorithm, which estimates the most probable sequence of states given a sequence of observations. The most likely path corresponds to the robot's trajectory, and we can use this to continuously update the robot's pose. The Forward Algorithm is used to compute the likelihood of different models, allowing the model to select the most suitable model for localization, maybe a more detailed map of a particular region, for example. This process is analogous to sub-maps in Cartographer, where each state is a centroid representing a region of free space, and the robot's odometry is reset whenever a transition between states occurs.

24.3.2 Training the Hidden Markov Model (HMM)

Training the HMM for SLAM involves learning the model parameters — the transition matrix A , the emission matrix B , and the initial state distribution π — from sensor data and robot movements. The goal is to estimate the most probable sequence of states (robot poses) based on the observed sensor measurements. In this section, we describe the training process, including how we handle sensor data, optimize model parameters, and update the state and observation models during exploration.

The Baum-Welch algorithm is widely used to estimate the parameters of Hidden Markov Models (HMMs) to maximize their alignment with a given training dataset. For computational efficiency, it employs the Forward Algorithm, a dynamic programming approach, to compute the likelihood of observations efficiently by breaking the problem into smaller overlapping subproblems. Without this approach, the naive computation of all possible state paths through the HMM would have a time complexity of $O(N^T)$, which is infeasible for realistic models. The Forward Algorithm reduces this to $O(N^2T)$ by recursively calculating probabilities, where N is the number of states and T is the length of the observation sequence. The forward variable $\alpha_t(i)$ represents the probability of observing the sequence $O_1, O_2, \dots, O_t$ and being in state i at time t . It is computed recursively as stated in Equation 24.7.

This computational efficiency enables the Baum-Welch algorithm and its variant, the Online Baum-Welch algorithm, to handle large datasets and complex models effectively.

$$\alpha_t(i) = \left[\sum_{j=1}^N \alpha_{t-1}(j) a_{ji} \right] b_i(O_t) \quad (24.7)$$

$$P(\vec{O} \mid \lambda) = \sum_{i \in \text{terminal}} \alpha_{t+1}(i) = \sum_{i=1}^N P(o_1, o_2, \dots, o_t, x_t = S_i \mid \lambda) \quad (24.8)$$

This technique is integral to both the Baum-Welch and its variant, the Online Baum-Welch algorithm.

Similarly, the auxiliary variable $\xi_{ij}(t)$ represents the probability of being in state i at time t and transitioning to state j at time $t+1$, given the observations $\vec{O}$ and the model λ . This is defined as:

$$\xi_{ij}(t) = P(S_t = i, S_{t+1} = j | \vec{O}, \lambda) = \frac{P(S_t = i, S_{t+1} = j, \vec{O} | \lambda)}{P(\vec{O} | \lambda)} \quad (24.9)$$

The denominator in (24.9) is the same as in the forward probability computation and can be expressed in terms of the forward (α) and backward (β) variables:

$$\xi_{ij}(t) = \frac{\alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)}{\sum_{i=1}^N \sum_{j=1}^N \alpha_t(i) a_{ij} b_j(o_{t+1}) \beta_{t+1}(j)} \quad (24.10)$$

The re-estimation formulas for the Baum-Welch algorithm are summarized as follows:

$$\gamma_i(t) = \frac{\alpha_t(i) \beta_t(i)}{\sum_{j=1}^N \alpha_t(j) \beta_t(j)} \quad (24.11)$$

$$\bar{\pi}_i = \gamma_i(1) = \frac{\alpha_1(i) \beta_1(i)}{\sum_{j=1}^N \alpha_1(j) \beta_1(j)} \quad (24.12)$$

$$\bar{a}_{ij} = \frac{\sum_{t=1}^{T-1} \xi_{ij}(t)}{\sum_{t=1}^{T-1} \gamma_i(t)} = \frac{\sum_{t=1}^{T-1} \alpha_t(i) a_{ij} b_j(o_t) \beta_{t+1}(j)}{\sum_{t=1}^{T-1} \alpha_t(i) \beta_t(i)} \quad (24.13)$$

$$\bar{b}_{jk} = \frac{\sum_{t=1}^{T-1} 1_{o_t=v_k} \gamma_j(t)}{\sum_{t=1}^{T-1} \gamma_j(t)} = \frac{\sum_{t \in O_t=v_k} \alpha_t(j) \beta_t(j)}{\sum_{t=1}^{T-1} \alpha_t(j) \beta_t(j)} \quad (24.14)$$

Equations 24.13 and 24.14 show an elegant and efficient method for estimating the values of the matrices directly from the training set. The function $1_{o_t=v_k}$ is a binary indicator function of the number of times v_k has been observed while being in state j .

24.3.2.1 Sensor Data Preprocessing

The first step in training is preprocessing the raw sensor data. This step includes:

- **Sensor Calibration:** Ensuring that sensor data is calibrated to provide accurate measurements of the robot's environment.
- **Quantization of Sensor Readings:** Sensor readings are discretized into a finite set of symbols through clustering (e.g., K-Means or Affinity Propagation [812]). This process transforms raw sensor data into discrete symbols that represent the state of the environment.

- **Sensor Fusion:** Combining data from multiple sensors (e.g., LiDAR, cameras, IMUs) into a unified representation [816]. Each sensor modality is associated with its own emission matrix. Fusion is achieved through a weighted average approach, where the weights are determined by the likelihood of each sensor's data. This likelihood serves as a measure of the confidence or reliability of the sensor in each specific scenario, ensuring an adaptive and robust fusion process.

This preprocessing allows the model to work with symbolic representations of sensor data, which simplifies the training process and makes the system adaptable to different types of sensors.

24.3.2.2 Model Initialization

We initialize the HMM model with the following steps:

- **Transition Matrix (A):** The transition probabilities between states are initially set based on prior knowledge of the robot's movement dynamics. For example, the probability of staying in the same state or transitioning to a neighboring state can be estimated based on motion constraints or odometry data.
- **Emission Matrix (B):** The emission probabilities are initialized based on a sample of sensor data, where each symbol corresponds to a cluster of sensor readings.
- **Initial State Distribution (π):** The initial distribution is set based on the robot's starting position or a uniform distribution if the starting position is unknown.

24.3.3 Model Improvement-OnLine Baum Welch Algorithm

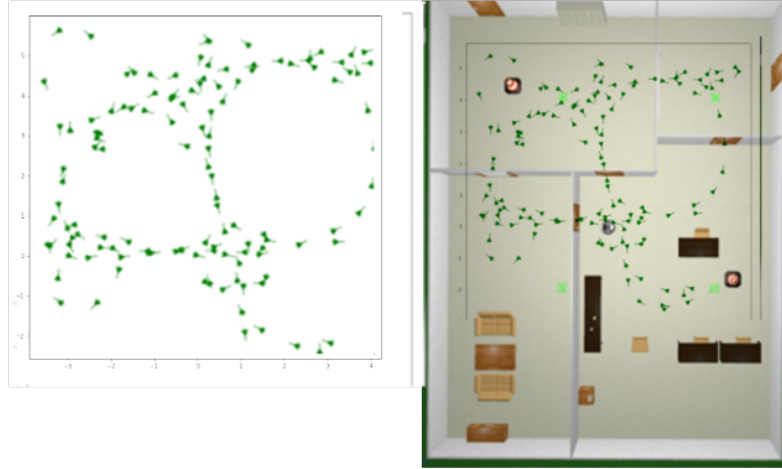
Online training plays a crucial role in improving the robot's topological representation of the environment. Using the Baum-Welch algorithm, the model parameters (Θ) are iteratively updated, scaled by a learning rate (η) in the direction of the gradient:

$$\theta^{(t+1)} = \theta^{(t)} - \eta \frac{\partial P(\vec{O} | \lambda)}{\partial \theta} \quad (24.15)$$

Here, $\Theta = \lambda$, and the parameters being updated are elements of the transition (A) and emission (B) matrices. The gradients are expressed as:

$$\frac{\partial P}{\partial a_{ij}} = \sum_{t=1}^{T-1} \alpha_t(i) b_j(O_{t+1}) \beta_{t+1}(j) \quad (24.16)$$

$$\frac{\partial P}{\partial b_j(O_t)} = \begin{cases} \delta_{j1} \beta_1(1), & t = 1 \\ \sum_{i=1}^N \alpha_{t-1}(i) a_{ij} b_j(O_{t+1}) \beta_t(j), & t \neq 1 \end{cases} \quad (24.17)$$

**FIGURE 24.3**

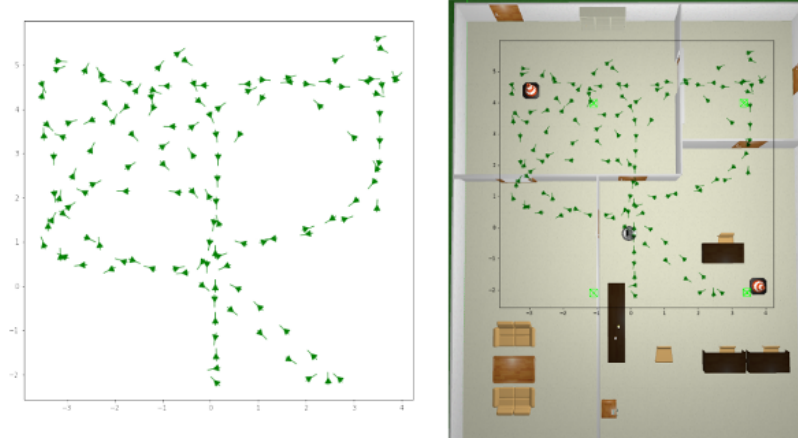
Topological representation obtained from offline training.

These expressions demonstrate that partial derivatives modify only specific elements of the matrices A and B . Infinitesimal adjustments are made per successful examples, guided by the gradient, which depends only on the state and observation being processed. Figures 24.3 and 24.4 illustrate how online updates refine the topological graph of the explored environment.

24.3.4 Benchmarking SLAM: Challenges and Considerations

The inherent complexity of the SLAM problem makes its evaluation particularly challenging. Existing benchmarking approaches, such as the metric proposed by Steder in [817], may not apply universally, especially to systems like ours that employ mapless representations. Evaluating SLAM requires considering multiple factors, including environmental variability and the specific characteristics of each segment within an environment. Certain conditions may favor one method over another, highlighting the importance of context-sensitive metrics.

Moreover, general purpose solutions require integrating diverse sensor data. Sensor fusion schemes, which combine readings from multiple modalities, consistently yield superior results. Evaluations must also account for variables such as task-specific demands, robot capabilities, lighting conditions, and even dynamic factors like the number of people in the environment. These factors emphasize the need for flexible benchmarking frameworks that address SLAM's diverse applications and operational contexts.

**FIGURE 24.4**

Adjusted representation after some successful runs have adjusted parameters

24.4 Experiments

This section details our experimental observations, focusing on benchmarking and evaluating the performance of our SLAM system. Particular emphasis is placed on identifying the conditions under which one sensor type (vision or LiDAR) outperforms the other. These observations are critical for understanding the robustness and adaptability of our method in diverse environments.

It is hypothesized that particle filters using well-defined maps perform optimally in static, feature-rich environments. Conversely, in low-feature scenarios, such as hallways, our method highlights the strength of vision-based feature extraction. This comparison underscores the complementary roles of LiDAR and vision sensors and the importance of choosing the appropriate technique for specific environmental challenges.

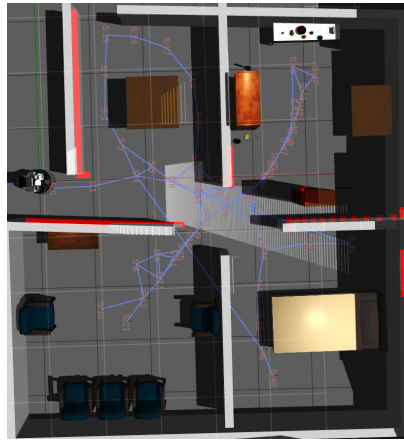
24.4.1 Training and Validation

The training was performed using wheel odometry over short episodes. Each episode terminated when the cumulative odometry error reached approximately three times the robot's radius, at which point the episode restarted. Although our previous work demonstrated an online training implementation, the results reported here focus on offline training for simplicity, knowing that the model will improve with successful episodes thanks to the Online Baum Welch implementation discussed above, see eq. 24.17.

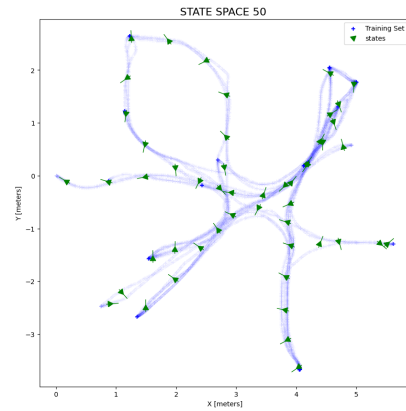
The training dataset, limited to 150 MB for our experiments, includes ap-

proximately 6,000 labeled examples of wheel odometry, in a Pose format, with whichever reading is being analyzed. The robot, in both real and simulated environments, roams freely by randomly selecting known locations commonly used in its tasks. Using offline Baum-Welch on the training set, an initial HMM is derived, from which the transition matrix is interpreted as the topological graph shown in Figures 24.5(a) and 24.5(b).

The robot's experience allows us to compute the centroids of the poses represented in the hidden state space, which are shown as green arrows in the same figure 24.5(b).



(a) Topological map obtained from the transition matrix superposed on the TMR2024 Mexico @Home arena.



(b) State space representing the training data.

FIGURE 24.5

Spatial representations used for learning and planning: (a) topological abstraction and (b) state space distribution.

For validation, the model was tested online as the robot navigated the environment using various strategies. These strategies included:

- **Human-controlled navigation,**
- **Potential field-based reactive navigation,**
- **Path planning,** and
- **Local map-based movement** generated from the LiDAR.

Pre-trained models from the offline phase were used for online pose estimation. We evaluated individual sensor symbol readings alongside the proposed fused model. Fusion during the experiments employed a likelihood-based weighted scheme, balancing contributions from vision and LiDAR. The likelihood from each model is obtained using the Forward Algorithm. This infor-

mation is used to compute the weighted sum proposed for fusing the model's symbols.

An example run consisting of 150 readings is shown in Figure 24.6. This example run is analyzed using each model without providing initial conditions. In other words, for each model, the states (represented by green arrows in the figure) most likely to have produced their own individual observation vectors are inferred. As shown in the figure, different models yield different estimates, represented by crosses of various colors. Finally, the fused model's output is represented by the large gray circles.

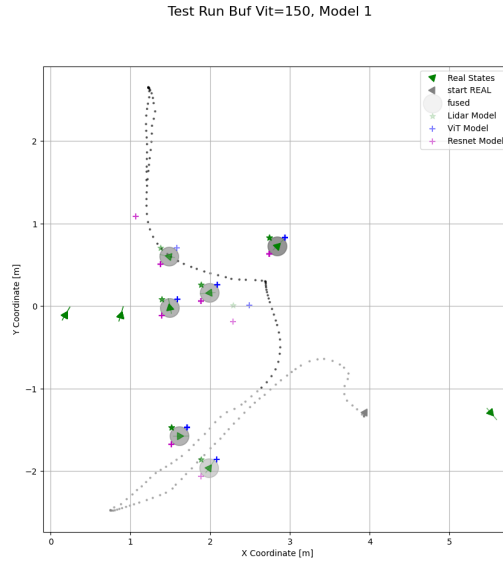


FIGURE 24.6

An example run showing all the individual models state space estimate, as well as the fused model.

24.4.2 Experimental Setup

The simulation environment was modeled as a typical apartment, resembling the setting used in the Mexican Robotics Tournament (TMR 2024). This environment spanned approximately 60 square meters and included a living room, kitchen, bedroom, and dining room. Using the HSR robot [818], the system autonomously explored the environment while performing its routine tasks (domestic tasks, in the case of the service robots discussed here) to generate the training dataset.

In real-world experiments, the robot demonstrated strong adaptability across diverse settings after careful system tuning. Tests conducted in the Bio Robotics Lab in Mexico, as well as the low-featured hallways leading to

it, highlighted the robustness of the proposed SLAM approach. Notably, the symbolic fusion method effectively handled environments with limited visual and structural features, where traditional vision-only systems often struggle. These real-world results validate the system's ability to transition seamlessly from simulation to practical deployment, reinforcing its versatility in dynamic and challenging scenarios.

24.4.3 Results

Initial fusion experiments utilized ResNet features. However, these were later outperformed by features extracted using a **DeiT Vision Transformer (ViT)**. Notably, the ViT feature vector size (126 dimensions) is significantly smaller than that of ResNet, offering computational efficiency alongside improved performance. This result highlights the potential of modern transformers in visual feature extraction for SLAM tasks. A new model fusing both vision based symbols with the LiDAR symbol showed similar performance, thus we conclude that the additional computational cost is not justified. The model called Fused in this section is formed by fusing the ViT based symbols with the LiDAR symbols.

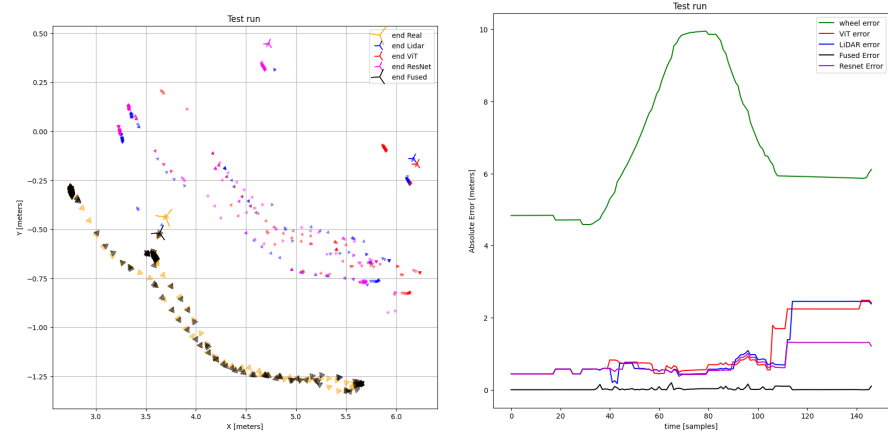
In our previous work [801], we proposed a more accurate method for converting state space estimates into odometry data. For evaluation purposes in this study, odometry adjustment is simplified by aligning to the midpoint between the last estimated states. During the first adjustment, only a single point is available, which increases the quantization error in the initial samples, especially in kidnapped robot scenarios.

To illustrate this, we include a graph comparing various odometry estimation methods alongside their respective absolute error plots in Figures 24.7(a) and 24.7(b). This comparison provides insight into the robustness and accuracy of our approach. In this particular run of 150 samples, we highlight a region where the fused model outperforms each of the individual models that compose it.

In Figure 24.7(b), it can be observed that the wheel odometry model has an initial error, which is a random value representing the kidnapped robot initial conditions. Around time sample 115 in the error plot, all individual models predict a state transition prematurely, increasing error. However, the fused model correctly estimates the transitions, maintaining lower error levels.

The following experiment demonstrates the performance of the fused model using our real robot, which typically relies on AMCL particle filtering for localization. A common challenge arises in the hallway leading to our lab, a real-world example of a low-feature environment for LiDAR. Such environments often cause faulty estimations when relying solely on LiDAR data. For this experiment, we trained the model under similar assumptions as those in the apartment experiment reported above. The topological representation found is shown in Figure 24.8(a).

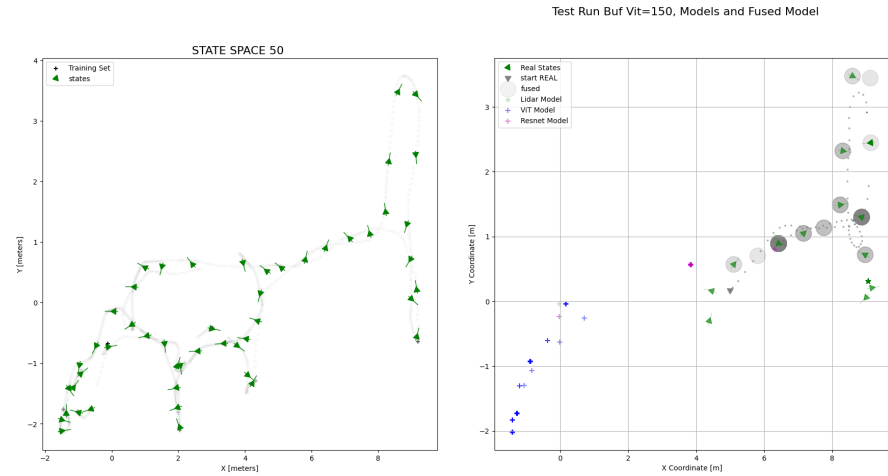
In the example run presented, the error peaks noticeably when the robot



(a) Different individual odometries vs the (b) Absolute error of odometries shown above.

FIGURE 24.7

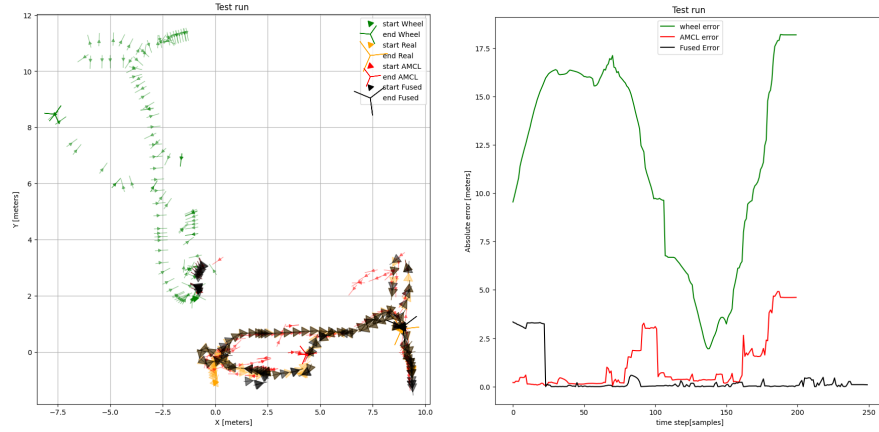
Comparison of multiple odometry models and their fused estimate: (a) trajectory overlay and (b) corresponding absolute error.



(a) State space representing the training data. (b) An example run showing all the individual models state space estimate in a low featured hallway.

FIGURE 24.8

State space visualizations: (a) training data representation and (b) example run of individual models' estimates.



(a) Different individual odometries vs the (b) Absolute error of odometries shown fused estimate.

FIGURE 24.9

Comparison of multiple odometry models and their fused estimate: (a) trajectory overlay and (b) corresponding absolute error.

enters the hallway, primarily due to the limitations of LiDAR in feature-sparse environments. As shown in Figure 24.9(a), the LiDAR-only model struggles to provide accurate state predictions in this region, resulting in significant localization errors. However, once the robot transitions back into the lab, the low-light conditions make vision-based methods unreliable, as illustrated in Figure 24.9(b). Despite these challenges, the fused model demonstrates its robustness by effectively handling both situations. The fused model successfully compensates for individual sensor failures, maintaining accurate localization throughout the run.

This experiment highlights the adaptability of the fused model in real-world scenarios, particularly in environments with varying sensory challenges.

24.5 Conclusions

In this chapter, we presented a novel approach to Simultaneous Localization and Mapping (SLAM) by integrating Hidden Markov Models (HMMs) with a symbolic fusion technique to combine visual and LiDAR sensor data. This lightweight, modular SLAM system enhances adaptability to dynamic and uncertain environments, demonstrating significant improvements over traditional sensor-based methods.

The proposed system stands out by employing a hybrid symbolic repre-

sensation, wherein sensor data is discretized into symbolic forms using vector quantization, enabling more efficient sensor fusion. By leveraging the flexibility of HMMs, our approach allows the dynamic adaptation of sensor configurations based on environmental conditions. This enables robust localization and mapping in scenarios where single-sensor approaches—particularly in challenging environments such as low-light or feature-sparse areas—often struggle.

One of the primary advantages of our system is its ability to seamlessly integrate multiple sensor modalities, such as, but not limited to, LiDAR and vision-based features, into a unified framework. In future work, it would be desirable to explore additional sensors, such as 3D point clouds, IMUs, and others. Our experiments highlight the complementary strengths of these sensors, enabling the system to mitigate individual sensor limitations and improve overall performance. Notably, our fusion approach dynamically adjusts the sensor contributions based on their reliability in a given region, further enhancing the robustness and accuracy of localization.

A significant contribution of this is the integration of the HMM framework with a probabilistic approach to sensor fusion. This approach not only improves the performance of the system in diverse environments but also lays the groundwork for more complex tasks, such as autonomous decision-making, by transforming the SLAM process into a Markov Decision Process (MDP). This opens the possibility for further exploration of reinforcement learning algorithms to optimize the SLAM system’s decision-making capabilities in real-time.

While the current system achieves robust localization across various conditions, several avenues for future work remain. First, real-time computational efficiency can be improved to allow for deployment in highly resource-constrained environments. Exploring the integration of additional sensor types, such as inertial measurement units (IMUs) or magnetic sensors, could provide even richer sensory data and further enhance the adaptability of the system. Additionally, more advanced reinforcement learning techniques could be incorporated to enable the system to optimize its performance autonomously over extended operational periods.

Furthermore, the scalability of the system to larger, more complex environments remains an area for exploration. As sensor fusion algorithms improve and more powerful computational resources become available, it will be possible to deploy this system in larger-scale indoor and outdoor environments, making it suitable for a wide range of autonomous navigation and robotic applications.

By integrating multiple sensor modalities through HMM-based symbolic fusion, the system provides a robust, adaptable, and computationally efficient solution. We believe that this work will contribute to the ongoing evolution of SLAM technologies, pushing the boundaries of what autonomous systems can achieve in diverse, real-world scenarios.

Acknowledgements

This research was supported by DGAPA UNAM: PAPIIT grant numbers IT101524 and IT102424.